Victor BOYADJIAN
Enys TCHENAR
Gaël BOTTON

RAPPORT TECHNIQUE

MSPR1 – HEALTH AI

Table des matières

CONTEXTE 3
Choix technologiques 4
Python 4
PHP 5
Docker 6
PostgreSQL 6
Grafana 7
Railway 7
Difficultés rencontrées 8
Problèmes de configuration 8
Complexité et ajustements 8
Résultats Obtenus 9
Perspectives d'évolution 10
Table des illustrations 11

CONTEXTE

Dans un contexte où les solutions numériques liées à la santé et au bien-être connaissent une croissance rapide, la startup HealthAI Coach souhaite se positionner comme un acteur innovant en proposant une plateforme complète de suivi personnalisé. Les utilisateurs sont de plus en plus demandeurs d'outils capables de centraliser leurs données (nutrition, activité physique, sommeil, biométrie) afin d'obtenir des recommandations adaptées à leurs objectifs. Cependant, ces données proviennent de sources multiples et hétérogènes (API, fichiers, open data), ce qui rend leur exploitation complexe sans une infrastructure adaptée. Il devient donc essentiel de mettre en place un système capable de collecter, structurer et fiabiliser ces données pour en tirer de la valeur.

C'est dans ce cadre que s'inscrit ce projet, dont l'objectif est de concevoir un backend métier robuste permettant d'automatiser l'ingestion, le nettoyage et le stockage des données, tout en les rendant exploitables via une API et des outils de visualisation. Cette solution constitue une brique fondamentale pour le développement futur de fonctionnalités avancées, notamment en intelligence artificielle. Elle doit également répondre à des enjeux industriels forts tels que la reproductibilité, la sécurité et la facilité de déploiement, afin de permettre aux équipes techniques de réutiliser et faire évoluer efficacement l'environnement mis en place.

Choix technologiques

Pour mener à bien ce projet, nous avons dû sélectionner différentes technologies en fonction des besoins, tout en prenant en compte la simplicité de mise en place et l'efficacité globale de la solution.

Python

En ce qui concerne le développement de l'ETL, nous avons décidé de partir sur le langage de programmation Python. Python est l'un des langages les plus utilisés au monde, avec une très grande communauté active, ce qui nous permet de travailler sans réinventer la roue à chaque fois, en utilisant des librairies et des frameworks fournis par celle-ci.

De plus, le but principal de cet ETL est de décortiquer tout type de données afin d'extraire les informations essentielles à l'alimentation de notre base de données. Python est le langage le plus utilisé dans le traitement de données, ce qui nous offre un large choix d'outils pour construire notre projet.

Dans le cadre du traitement de données avec Python, nous utilisons la librairie Pandas, qui facilite grandement la lecture et la manipulation de fichiers de données tels que les formats CSV, XLSX, etc.

Enfin, pour des raisons de performance et de simplicité, nous avons décidé de développer nous-mêmes notre ETL, car celui-ci sera exactement adapté à nos besoins, sans fournir trop peu de fonctionnalités ni en proposer inutilement.

Figure 1 : Logo de Python

PHP

Pour ce qui est de l'API, nous avons utilisé le langage de programmation PHP combiné au framework Laravel. Laravel est un framework backend open source permettant de créer des API performantes et structurées. Nous avons choisi cette technologie car il s'agit d'un environnement que nous maîtrisons, ce qui nous permet de développer une API solide, sécurisée et stable.

De plus, PHP, tout comme Python, bénéficie d'une large communauté proposant de nombreuses librairies, ce qui facilite grandement le développement et nous évite de réinventer certaines fonctionnalités.

Concernant le fonctionnement de l'API, nous avons utilisé l'ORM de Laravel, qui permet d'interagir avec la base de données via des classes appelées "Models". Chaque modèle est associé à une table correspondante, ce qui simplifie la gestion des données et les opérations CRUD en base de données.

Le framework propose également de nombreux outils intégrés, tels que la gestion des routes, l'authentification, la validation des données ou encore la gestion des erreurs, ce qui permet de gagner du temps lors du développement tout en respectant les bonnes pratiques. De plus, Laravel intègre nativement des mécanismes de sécurité comme la protection contre les injections SQL, les attaques CSRF ou encore le chiffrement des données sensibles, ce qui renforce la fiabilité de l'API.

Enfin, nous avons également fait ce choix technologique car, chez XEFI, nous utilisons une librairie développée en interne par notre directeur technique. Celle-ci permet de faciliter la création des routes d'API ainsi que leur sécurisation, ce qui constitue un avantage supplémentaire dans le cadre de ce projet.

Figure 3 : Logo PHP

Figure 2 : Logo Laravel

Docker

Pour le bon fonctionnement du projet, nous avons utilisé **Docker**, une technologie de conteneurisation permettant d'isoler chaque service dans un environnement indépendant appelé conteneur. Cette approche garantit une meilleure séparation des responsabilités entre les différents composants de l'application (ETL, API, base de données, etc.), tout en assurant une cohérence globale du système.

Docker permet de créer des **images** reproductibles, qui contiennent l'ensemble des dépendances et configurations nécessaires à l'exécution d'un service. Ces images peuvent ensuite être déployées facilement sur différents environnements (RD, RD, PROD) ce qui réduit considérablement les problèmes liés aux environnements.

Dans notre architecture, chaque service est encapsulé dans un conteneur dédié, ce qui facilite la maintenance, la scalabilité et la mise à jour indépendante des composants. Cette modularité permet également d'éviter les conflits de dépendances entre les services.

Figure 4 : logo Docker

PostgreSQL

Pour ce qui est de la base de données, nous avons utilisé PostgreSQL. PostgreSQL est un moteur de base de données relationnelle (SQL) puissant, reconnu pour sa robustesse, sa fiabilité et sa capacité à gérer des volumes importants de données tout en garantissant leur intégrité.

Nous avons choisi ce système de gestion de base de données car il offre d'excellentes performances, une grande stabilité, ainsi qu'une forte capacité de scalabilité. Il est particulièrement adapté aux applications nécessitant un traitement structuré et sécurisé des données, comme c'est le cas dans notre projet HealthAI Coach.

De plus, le framework Laravel utilise des bases de données relationnelles pour fonctionner. C'est pour cela que nous avons choisi PostgreSQL, car il répond exactement aux prérequis du framework et est également un moteur SQL que nous maîtrisons.

Figure 5 : logo PostgreSQL

Grafana

Dans le cadre du suivi et de la visualisation des données, nous avons utilisé Grafana, une plateforme open source spécialisée dans la création de tableaux de bord interactifs.

Grafana permet de connecter différentes sources de données (comme PostgreSQL) afin de construire des visualisations dynamiques et personnalisées. Grâce à cet outil, nous pouvons suivre en temps réel les métriques clés de notre application, telles que les performances, l'évolution des données utilisateurs ou encore l'activité du système.

Figure 6 : logo Grafana

Railway

Pour le déploiement et l'hébergement de notre application, nous avons choisi la plateforme Railway.

Railway est une plateforme cloud qui simplifie grandement le déploiement d'applications en automatisant la CI/CD en quelques clicks. Elle permet de déployer rapidement des services (API, base de données, interface web, etc.) à partir de notre code source, sans nécessiter une configuration complexe de serveurs.

L'un des principaux avantages de Railway est sa compatibilité avec Docker, ce qui nous permet de déployer directement nos conteneurs dans un environnement cloud cohérent avec notre architecture locale. Cela garantit une continuité entre les environnements de développement et de production. Railwai permet également de choisir la position géographique de l'ébergement, ainsi que de dupliquer un même service à plusieurs endroits du monde différents.

Figure 7 : logo Railway

Difficultés rencontrées

Au cours du développement du projet, nous avons rencontré plusieurs types de problématiques, qu'elles soient liées au développement fonctionnel de l'application ou à la configuration de l'environnement technique.

Problèmes de configuration

Une des principales difficultés rencontrées concernait la mise en place de plusieurs schémas SQL distincts au sein de la base de données. L'objectif était de séparer clairement les données système des données métier afin d'éviter toute confusion et de mieux organiser la structure globale.

Cette séparation nécessitait une configuration précise de Laravel et de PostgreSQL, afin de leur indiquer correctement quel schéma devait être utilisé pour chaque type de données. L'enjeu principal était d'éviter toute pollution de la base de données en garantissant une stricte séparation des responsabilités.

Nous avons mis en place deux schémas :

- **System**, regroupant les tables internes utilisées par Laravel (telles que les tokens d'authentification, le cache, les jobs ou encore les logs de tâches échouées),
- **Data**, dédié aux données métiers utilisés par l'API et exploitables par les utilisateurs.

Cette organisation implique une gestion particulière des migrations. Les migrations liées au schéma system sont exécutées directement par Laravel, car elles font partie de son fonctionnement interne. En revanche, les migrations du schéma data, qui concernent les tables métiers, ont été externalisées afin de ne pas dépendre directement de Laravel, celui-ci n'ayant pas vocation à gérer ce type de structure mais simplement à les utiliser.

Pour gérer, les migrations du schéma Data, nous avons utilisé Flyway, un outil spécialisé dans la gestion et le versioning des migrations SQL. Flyway nous permet de maintenir un contrôle sur l'évolution de la base de données et d'appliquer les scripts SQL de manière ordonnée et reproductible.

Complexité et ajustements

La mise en place de cette architecture a nécessité un temps de configuration et de débogage important. En effet, même en fin de développement, nous avons rencontré des problèmes liés à la gestion des schémas, avec des interférences entre les différentes parties de la base de données.

Ces difficultés nous ont obligés intervenir directement dans les configurations de Laravel, PostgreSQL et Flyway afin d'assurer un comportement cohérent du système. Des ajustements ont été nécessaires pour garantir que chaque composant accède uniquement aux données qui lui sont destinées.

Malgré ces contraintes, cette approche nous a permis d'obtenir une architecture plus propre, plus modulaire et plus robuste.

Coordination de l'équipe et planification

Comme chacun des membres de l'équipe travaillent sur des sujets aussi complexes que dépendant les uns des autres, cela a d'abord pris du temps pour plannifier l'ensemble des taches du projet, puis il a été parfois difficile de respecter les délais fixé à chaque tache pour ne pas perturber l'avancée du projet.

Mettre en place une CI/CD automatisée nous a permis de fluidifier grandement l'avancée du projet, car les développements effectués étaient directement déployés et accessibles à tous via internet, donc pas besoin de faire tourner tous les différents services du projet sur une machine pour pouvoir développer un autre qui les utilise.

Résultats Obtenus

Grâce à la mise en place de l'ensemble de l'architecture du projet, plusieurs résultats concrets ont été atteints.

Tout d'abord, nous avons développé une API fonctionnelle et complète, intégrant tous les modèles définis initialement. Cette API repose sur des opérations CRUD (Create, Read, Update, Delete), permettant de manipuler efficacement les données. De plus, un système de gestion des rôles, des politiques et des permissions a été mis en place afin de sécuriser les accès et contrôler précisément les actions des utilisateurs.

Ensuite, le projet dispose désormais d'un système de migration opérationnel et fiable, garantissant la cohérence et l'évolution de la base de données au fil des modifications du projet. Cela permet d'appliquer les changements de structure sans perte de données et de maintenir une base toujours à jour.

Un pipeline ETL (Extract, Transform, Load) a également été implémenté et fonctionne correctement. Il permet d'extraire les données, de les transformer selon les besoins métier, puis de les charger dans le système cible de manière automatisée.

Par ailleurs, les environnements Docker ont été mis en place avec succès. Ils sont stables, reproductibles et permettent de lancer rapidement l'ensemble du projet, que ce soit en développement, en test ou en production. Cela assure une grande cohérence entre les différents environnements.

Enfin, une solution de monitoring a été intégrée avec Grafana. Un tableau de bord a été configuré pour afficher des métriques en temps réel, permettant de suivre l'état du système, les performances et d'identifier rapidement d'éventuels problèmes.

Perspectives d’évolution

Une perspective d’évolution la plus logique avec une architecture entièrement dockerisée serait d’améliorer la haute disponibilité de la solution.

Docker permet la duplication rapide d’un même service permettant de gérer les fortes affluences. Et l’outil d’hébergement, et de déploiement automatique Railway nous permettra de dupliquer un même conteneur, répondant au même domaine, mais à des emplacements géographiques différent grâce au load-balancing, pour une évolution à l’international.

Pour améliorer la haute disponibilité il sera également indispensable de mettre en place des sauvegardes régulières et automatisées de la base de données.

Table des illustrations

Figure 1 : Logo de Python 4
Figure 2 : Logo Laravel 5
Figure 3 : Logo PHP 5
Figure 4 : logo Docker 6
Figure 5 : logo PostgreSQL 6
Figure 6 : logo Grafana 7
Figure 7 : logo Railway 7